LOKI: An LLM Accelerator with Optimized KV Cache and In-Memory Computing Cores

Avadh Harkishanka, Abhishek Tyagi, Xuanyao Fong
Department of Electrical Engineering, National University of Singapore, Singapore
Email: avadh@u.nus.edu, atyagi@u.nus.edu, kelvin.xy.fong@nus.edu.sg

2026 IEEE International Symposium on Circuits and Systems (ISCAS) | 979-8-3315-7769-8/26/$31.00 ©2026 IEEE | DOI: 10.1109/ISCAS66217.2026.11562728

***Abstract*—Large language models (LLMs) are scaling rapidly, both in model size and context length, leading to rapidly increasing compute and memory demands during inference. The quadratic growth of attention mechanisms with sequence length imposes heavy key-value (KV) cache storage and bandwidth requirements, which dominate inference costs. Efficient KV cache management has thus become crucial for scalable and energy-efficient LLM deployment on hardware. This paper presents LOKI—a network-on-chip (NoC)-based accelerator architecture designed to address these challenges using system-level KV cache distribution and importance-driven KV cache pruning. Each processing element (PE) integrates an in-memory computing (IMC) core for weight-stationary (WS) matrix operations, digital attention blocks for pruned attention computation, and scratchpad memories for distributed KV cache storage. A dual-mode dataflow adapts to the distinct computational needs of the prefill and decode phases. Experimental results show that LOKI achieves 155.8× higher energy efficiency and 1.87× greater throughput compared to NVIDIA A100 GPU.**

***Index Terms*—LLM accelerator, KV cache pruning, In-Memory Computing, Prefill-Decode Optimization**

I. Introduction

Recent advances in LLMs have driven rapid growth in model size and context length to improve performance in tasks such as document summarization and code generation [1], [2]. However, this scaling demands massive parallel computation and frequent memory accesses, exposing severe memory wall bottlenecks in existing CPU and GPU architectures. The consequences are high latency and energy consumption (particularly for model weights and KV caches) during inference [3].

IMC offers a promising hardware alternative that significantly reduces data movement by performing computation directly within memory arrays. Its WS approach enables efficient matrix-vector multiplication (MVM) and multiply-accumulate (MAC) operations with higher throughput and lower energy usage. Recent IMC-based accelerators that leverage emerging nonvolatile memory (eNVM), such as resistive RAM (RRAM), have demonstrated substantial efficiency gains for deep neural network workloads, including LLMs [3]–[5]. However, innovations in accelerator architecture are needed to overcome the limitations of RRAM IMC technology.

LLM inference consists of two phases: prefill and decode. The *prefill* phase involves highly parallel computation to process input tokens and populate the KV cache. In contrast, the *decode* phase is sequential and memory-bound, requiring repeated accesses to the growing KV cache for each generated token [6]. While existing accelerators are typically optimized for the parallel prefill phase, the decode phase remains a major bottleneck due to limited memory bandwidth and reduced parallelism [5], [7], [8].

The increase in KV cache size with each generated token fundamentally limits the maximum context length and introduces memory bound delays during the decode phase [9], [10]. To address this challenge, recent research explored KV cache pruning techniques that take advantage of the inherent sparsity observed in attention maps to discard redundant tokens [11]–[14]. These strategies reduce memory and bandwidth demands without having a significant impact on model accuracy.

IMC-based LLM accelerators, while promising for reducing data movement, have particularly suffered from KV cache explosion due to their limited on-chip memory capacity [7]. Some recent works have attempted to integrate KV cache pruning with IMC architectures to exploit local data reuse and minimize off-chip memory accesses [15]–[17]. However, these designs remain constrained by limited scalability, rigid cache partitioning, and lack comprehensive system-level integration, rendering them impractical for accelerating large-scale LLMs.

In this work, we introduce LOKI, an LLM accelerator architecture that addresses the dual challenges of compute efficiency and memory scalability in LLM inference. LOKI combines: i) a system level KV cache allocation and routing strategy that improves bandwidth utilization and alleviates decode phase bottlenecks; and ii) a lightweight Pruning Engine (PNE) that dynamically retains tokens based on their importance scores without any latency overhead. Our results show LOKI achieves 1.87× throughput improvement and 155.8× energy reduction compared to the NVIDIA A100 GPU.

II. LOKI Architecture

A. Core Architecture

As illustrated in Fig. 1(a), *LOKI* is a heterogeneous LLM accelerator with PEs arranged on a 2D-mesh NoC. We employ two types of PEs: Matrix-Multiplication (MM) PEs (Fig. 1(c)) and KV PEs (Fig. 1(d)). MM PEs execute the heavy, static-weight general matrix-multiplications (GEMMs) for W_Q, W_O, and the multi-layer perceptron (MLP) projections. Each MM PE consists of two RRAM-based IMC crossbar arrays (64 KB each) and executes WS matrix operations that reduce

This work is funded in part by the National University of Singapore through the Microelectronics Seed Grant (FY2024); and in part by the National Research Foundation (NRF), Singapore, under the Competitive Research Programme (Award NRF-CRP24-2020-0002 and NRF-CRP24-2020-0003).

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 30,2026 at 04:16:59 UTC from IEEE Xplore. Restrictions apply.

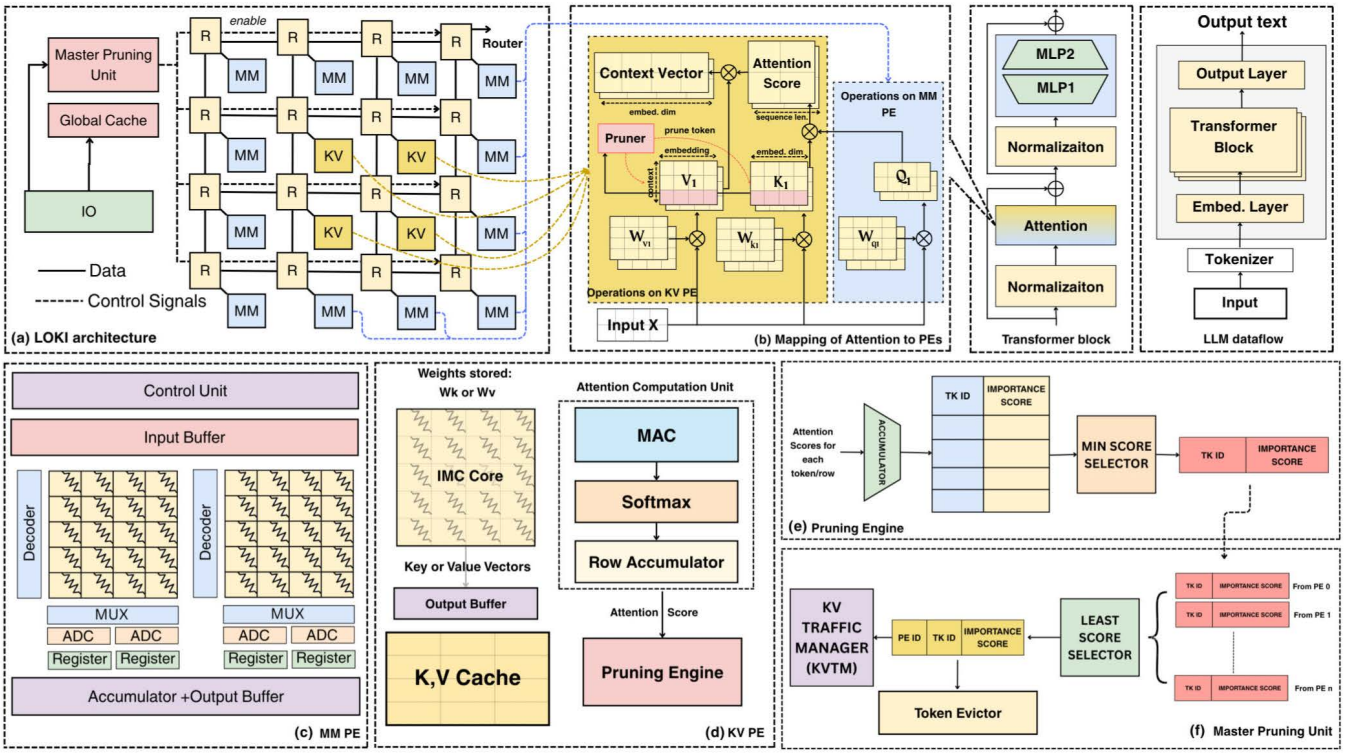


Fig. 1: Architecture and dataflow of LOKI. (a) LOKI architecture, (b) Breakdown of attention block and its mapping onto hardware PEs, (c) MM PE, (d) KV PE, (e) Pruning Engine, (f) Master Pruning Unit.

data movement and mitigate memory-wall overhead. KV PEs generate and store keys and values with each KV PE including one 64 KB IMC crossbar to realize the W_K/W_V projections, a digital attention compute block (for dynamic MAC, softmax and row aggregation), distributed KV cache (16 KB on each PE), and a lightweight PNE that ranks tokens by a simple importance score and evicts the least important entries. With temporal scheduling and pipelining, the NoC parallelizes processing and overlaps communication delay with computation, thereby minimizing the NoC-induced delay.

B. Mapping & Splitting

The weight matrices typically exceed the capacity of a single RRAM-based IMC core. Hence, each matrix is split along rows and columns to fit the available crossbar dimensions. As shown in Fig. 1 (b), MM PEs host W_Q, W_O, and MLP projections, while KV PEs store the W_K and W_V to generate keys and values in-place.

After splitting the layers across PE, they are allocated on the 2D mesh NoC to minimize on-chip traffic and latency. The projection layers (W_Q, W_K, W_V, W_O) and the MLP weights use a WS mapping where the weights remain resident in IMC while activations stream through. For the attention block, we adopt the output stationary (OS) mapping to keep the produced keys and values locally within each KV PE. During decode, queries are multicast to the KV PEs and attention executes near the data with only partial reductions traversing the network. The placement is *NoC-aware*—the mapping engine analyzes data dependencies and estimated traffic to co-locate the most communication intensive producer-consumer pairs (*e.g.*, W_Q output sources near the KV PEs) thereby reducing the hop count and latency.

C. Prefill Dataflow

During *prefill*, a large batch of tokens enters the accelerator and inputs are broadcast over the NoC to compute Q, K, and V in parallel. Whereas MM PEs produce Q, KV PEs, generate K and V using their local IMC cores and write them into the distributed KV cache. Each KV PE computes local $QK^\top$ with its digital MAC, applies on-chip softmax [18] to obtain the attention weights. The KV PEs that generate K also compute the attention weights $A = \mathrm{softmax}(QK^\top)$. These weights are broadcast to the KV PEs that hold V to form the context vector, which is then multicast to the MM PEs for calculating the W_O projection and the subsequent MLP. If the number of incoming tokens exceeds the KV cache capacity, the *PNE* activates on the write path, keeping only the most important tokens, ranked by the importance score, so that the cache remains bounded without any overhead.

D. Decode Dataflow

The decode stage operates sequentially, processing one token at a time. As it is memory bound, the expansion of the KV cache with each generated token leads to increasing

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 30,2026 at 04:16:59 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: KV Cache Pruning Algorithm.

Input: Number of input tokens S, KV cache capacity M

Output: Updated KV cache with pruned tokens

if $S < M$ **then**

- Store new input token in an empty cache slot;
- Update importance scores for all tokens;
- Identify least important token $\arg\min_t \mathcal{I}(t)$;

else if $S \geq M$ **then**

- Evict token with the minimum importance score $\arg\min_t \mathcal{I}(t)$;
- Store new input token in the freed cache slot;
- Compute importance score $\mathcal{I}(t_{new})$ for the new token;
- Update importance scores for all tokens;
- Re-identify least important token $\arg\min_t \mathcal{I}(t)$;

latency in prior designs [7], [9]. LOKI bounds the cache size using the PNE, reducing memory access time and energy per decode iteration. A distributed KV cache enables parallel reads and near data attention across cores, while the dataflow adapts to single token input and multicasts it to maximize parallelism, further reducing decode latency.

III. KV Cache Pruning

A. KV Cache Pruning Algorithm

The proposed KV cache pruning algorithm leverages the concept of an *importance score* to decide which tokens to retain or evict, subject to the KV cache memory capacity. Each token is assigned an importance score that reflects its contextual relevance. The score is computed as the cumulative sum of its attention weights in prefill and all decoding steps. The pruning mechanism uses the importance score as the eviction criterion. When the cache is full, the token with the lowest importance score is discarded to make space for the new token and thereby ensuring that the cache maintains only the most critical tokens for efficient decoding. The KV cache pruning algorithm is summarized in Algorithm 1.

B. Pruning Engine (PNE)

The PNE in Fig. 1(e) maintains token importance for the KV pruning algorithm by accumulating the attention weights and updating the scores with each new token as shown in Fig. 2. To avoid performance bottlenecks, the PNE operates in the background during both the prefill and decode phases to ensure that no additional latency is introduced into the token generation pipeline. Each PE records its least-important token and reports it to the *master pruning unit* (MPNU), which consolidates this information across all PEs. The MPNU as shown in Fig. 1(f) identifies the globally least important token and issues an eviction command when the KV cache reaches its maximum capacity, M. The new token is then written into the vacated slot, with the MPNU managing the dataflow to ensure a seamless update. During prefill, over-capacity cases

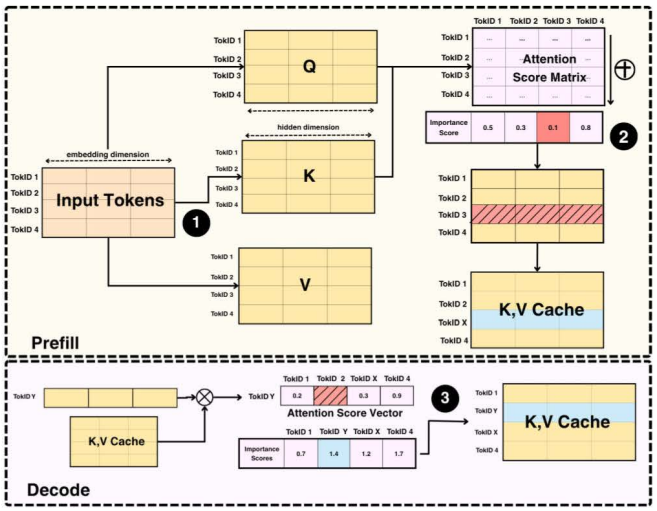


Fig. 2: KV cache pruning workflow: (1) if $S < M$, write K and V to the cache; (2) if $S \geq M$, evict the lowest-importance token; (3) during decode, insert the new token into KV cache.

TABLE I: Area and power breakdown.

Circuit	Area (mm^2)	Power (mW)
MM PE	0.1016	0.122
KV PE (total)	0.0964	0.198
IMC core	0.0508	0.061
Attention block	0.0408	0.012
KV cache	0.0046	0.042
Pruning Engine	0.0002	0.083

are precomputed so eviction can start at every decode step with no runtime overhead.

IV. Experimental Results and Discussion

A. Experimental Setup

To evaluate the proposed KV cache pruning and accelerator design, we implemented the PE design using a combination of digital and RRAM-based IMC crossbars. The attention block, consisting of digital components, was implemented in Verilog HDL and synthesized using Synopsys Design Compiler with a 45 nm standard cell library [19]. The scratchpad memory for the KV cache parameters was modeled using CACTI [20]–[22] at the 45 nm technology node.

For a fair comparison with state-of-the-art accelerators and NVIDIA GPUs, which are based on a 7 nm process node [23], all reported results in Table I are scaled to 7 nm. The design of the RRAM-based IMC core was adopted from [24].

A comparison of LOKI with state-of-the-art LLM accelerators is presented in Table II. Existing IMC-based LLM accelerators optimize MAC operations but do not efficiently address KV cache growth to ensure scalable decode performance for long context lengths.

B. Impact of distributed KV cache and Pruning

This section evaluates how the proposed KV cache optimization affects throughput, energy efficiency, accuracy, and area as shown in Fig. 3.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 30,2026 at 04:16:59 UTC from IEEE Xplore. Restrictions apply.

TABLE II: Comparison with Other IMC-based LLM Accelerators.

LLM accelerator	Technology	KV cache distribution	Pruning technique	Long-context decode scalability
ReTransformer [5]	RRAM IMC	No KV management	None	Limited (linearly growing KV)
Titanus [17]	SRAM digital IMC	Off-chip KV cache	Layer-wise magnitude thresholding	Moderate (off-chip KV)
CIMFormer [16]	Digital SRAM IMC	No dedicated KV caching	Attention based Importance Score	Moderate(sublinearly growing KV)
LEAP [7]	Hybrid RRAM IMC	Inside NoC router	None	Limited (linearly growing KV)
LOKI (this work)	Hybrid RRAM IMC	*Distributed* across KV PEs	Cumulative Attention based Importance Score	Scalable (*bounded* KV)

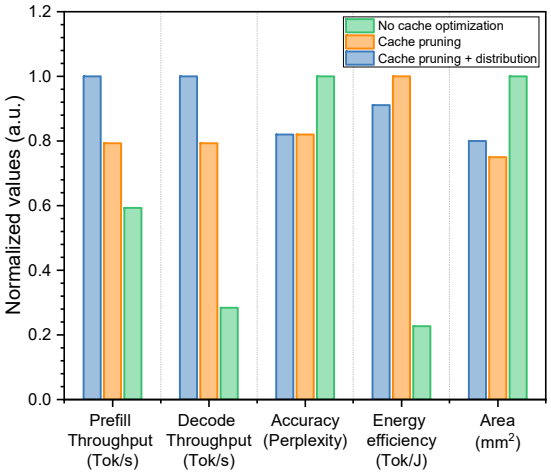


Fig. 3: Normalized impact of KV-cache pruning and distribution on prefill and decode throughput, energy efficiency, area, and accuracy for LLaMA-13B(sequence length 2048).

1) Throughput: During prefill, pruning low utility tokens reduces KV cache build time. Fewer tokens reach downstream layers of the LLM, which reduces computation and on-chip traffic and hence, increases throughput as illustrated in Fig. 3. Co-locating the distributed KV cache with IMC crossbars in KV PEs further reduces NoC traffic and improves throughput.

During decode, a bounded cache yields similar per token latency for all decode iterations. The distributed KV cache enables parallel reads and computation across PEs, and together with shorter access paths, increases decode throughput. Moreover, the PNE selects the eviction target before each decode iteration without any latency overhead.

2) Energy: Pruning the KV cache reduces token traffic and compute load by filtering low utility tokens early, thereby eliminating their downstream computations and reducing the energy consumption. It also reduces KV writes during prefill and KV reads at each decode step. However, distributing the KV cache across PEs increases the number of hops and hence, the interconnect energy.

3) Accuracy: We evaluate the impact of pruning on accuracy for *LLaMA-13B* at a sequence length of 2,048 using WIKITEXT-2 [25]. Perplexity increases from 8.57 to 10.21 after pruning, which is a modest degradation in accuracy.

4) Area and Memory: By retaining only high utility keys and values, the required KV capacity scales with the retained set rather than with sequence length, thereby reducing cache size by $2\times$–$4\times$ [12]. The PNE is compact, introducing negligible area overhead relative to the cache reduction. Distributing the KV cache across PEs marginally increases the area due to the added peripheral logic per block.

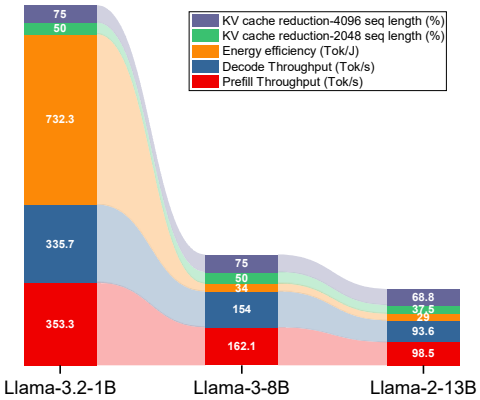


Fig. 4: Prefill/decode throughput, energy efficiency, and KV cache reduction across sequence lengths for *LLaMA* models.

TABLE III: Performance Comparison with NVIDIA GPUs

Hardware	Throughput (tokens/s)	Energy Efficiency (tokens/J)
A100	52.6	0.18
H100	151.45	0.32
LOKI	98.53	28.98

All results correspond to LLaMA-13B with a 2,048-token context. LOKI metrics are projected to 7 nm node for fair, process-matched comparison.

Fig. 4 presents a system-level performance comparison and KV cache reduction for *LLaMA* [26] across varying model sizes and sequence lengths. For *LLaMA-13B* with context length of 2048, KV cache pruning reduces the per-layer cache requirement to 62.5% of the original size.

Table III reports that LOKI attains $1.87\times$ higher throughput and $155.8\times$ greater energy efficiency than A100 for *LLaMA-13B* model. While LOKI achieves lower throughput than H100, it is $90.6\times$ more energy efficient.

V. CONCLUSION

LOKI bounds KV cache growth and sustains throughput, enabling scalable deployment of IMC accelerators for modern LLM workloads. By combining distributed cache placement with importance driven pruning, LOKI effectively mitigates the memory bound bottleneck of long context inference while preserving accuracy. Experimental evaluations demonstrate that LOKI enables high throughput, scalable, and energy efficient prefill and decode execution for LLMs.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 30,2026 at 04:16:59 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] B. Szalontai, G. Szalay, T. Márton, A. Sike, B. Pintér, and T. Gregorics, "Large language models for code summarization," 2024. [Online]. Available: https://arxiv.org/abs/2405.19032

[2] M. Ravaut, A. Sun, N. Chen, and S. Joty, "On context utilization in summarization with large language models," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, L.-W. Ku, A. Martins, and V. Srikumar, Eds. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 2764–2781. [Online]. Available: https://aclanthology.org/2024.acl-long.153/

[3] N. Leroux, P. Manea, C. Sudarshan *et al.*, "Analog in-memory computing attention mechanism for fast and energy-efficient large language models," *Nature Computational Science*, vol. 5, pp. 813–824, 2025. [Online]. Available: https://doi.org/10.1038/s43588-025-00854-1

[4] L. Xu, S. Yuan, D. Wang, Y. Chen, X. Li, and Y. Sun, "Heirs: Hybrid three-dimension rram- and sram-cim architecture for multi-task transformer acceleration," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, ser. DAC '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: https://doi.org/10.1145/3649329.3657327

[5] X. Yang, B. Yan, H. Li, and Y. Chen, "Retransformer: Reram-based processing-in-memory architecture for transformer acceleration," in *2020 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, 2020, pp. 1–9.

[6] A. K. Kamath, R. Prabhu, J. Mohan, S. Peter, R. Ramjee, and A. Panwar, *POD-Attention: Unlocking Full Prefill-Decode Overlap for Faster LLM Inference*. New York, NY, USA: Association for Computing Machinery, 2025, p. 897–912. [Online]. Available: https://doi.org/10.1145/3676641.3715996

[7] Y. Wang, Y. J. Chong, and X. Fong, "LEAP: LLM inference on scalable PIM-NoC architecture with balanced dataflow and fine-grained parallelism," in *2025 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 2025, pp. 1–9.

[8] C. Duan, J. Yang, R. Yang, Y. Wang, Y. Wang, L. Long, Y. Qi, X. He, A. Zhou, X. Wang, and W. Zhao, "Hpim: Heterogeneous processing-in-memory-based accelerator for large language models inference," 2025. [Online]. Available: https://arxiv.org/abs/2509.12993

[9] R. Geens, M. Shi, A. Symons, C. Fang, and M. Verhelst, "Energy cost modelling for optimizing large language model inference on hardware accelerators," in *2024 IEEE 37th International System-on-Chip Conference (SOCC)*, 2024, pp. 1–6.

[10] A. Łańcucki, K. Staniszewski, P. Nawrot, and E. M. Ponti, "Inference-time hyper-scaling with kv cache compression," 2025. [Online]. Available: https://arxiv.org/abs/2506.05345

[11] D. Yang, X. Han, Y. Gao, Y. Hu, S. Zhang, and H. Zhao, "Pyramidinfer: Pyramid kv cache compression for high-throughput llm inference," 2024. [Online]. Available: https://arxiv.org/abs/2405.12532

[12] Q. Fu, M. Cho, T. Merth, S. Mehta, M. Rastegari, and M. Najibi, "Lazyllm: Dynamic token pruning for efficient long context llm inference," 2024. [Online]. Available: https://arxiv.org/abs/2407.14057

[13] S. G. S, S. Prakash, and B. Ravindran, "Aqua: Attention via query magnitudes for memory and compute efficient inference in llms," 2025. [Online]. Available: https://arxiv.org/abs/2509.11155

[14] R. Ghadia, A. Kumar, G. Jain, P. Nair, and P. Das, "Dialogue without limits: Constant-sized kv caches for extended responses in llms," 2025. [Online]. Available: https://arxiv.org/abs/2503.00979

[15] W. Xu, W. Zeng, Q. Huang, M. Li, and R. Huang, "Unicaim: A unified cam/cim architecture with static-dynamic kv cache pruning for efficient long-context llm inference," 2025. [Online]. Available: https://arxiv.org/abs/2504.07479

[16] R. Guo, X. Chen, L. Wang, Y. Wang, H. Sun, J. Wei, H. Han, L. Liu, S. Wei, Y. Hu, and S. Yin, "Cimformer: A systolic cim-array-based transformer accelerator with token-pruning-aware attention reformulating and principal possibility gathering," *IEEE Journal of Solid-State Circuits*, vol. 59, no. 10, pp. 3317–3329, 2024.

[17] P. Chen and X. Yang, "Titanus: Enabling kv cache pruning and quantization on-the-fly for llm acceleration," in *Proceedings of the Great Lakes Symposium on VLSI 2025*, ser. GLSVLSI '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 71–77. [Online]. Available: https://doi-org.libproxy1.nus.edu.sg/10.1145/3716368.3735145

[18] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: Fast and memory-efficient exact attention with io-awareness," 2022. [Online]. Available: https://arxiv.org/abs/2205.14135

[19] J. E. Stine, I. Castellanos, M. Wood, J. Henson, F. Love, W. R. Davis, P. D. Franzon, M. Bucher, S. Basavarajaiah, J. Oh, and R. Jenkal, "Freepdk: An open-source variation-aware design kit," in *2007 IEEE International Conference on Microelectronic Systems Education (MSE'07)*, 2007, pp. 173–174.

[20] N. Muralimanohar, R. Balasubramonian, and N. Jouppi, "Optimizing nuca organizations and wiring alternatives for large caches with cacti 6.0," in *40th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO 2007)*, 2007, pp. 3–14.

[21] S. Thoziyoor, J. H. Ahn, M. Monchiero, J. B. Brockman, and N. P. Jouppi, "A comprehensive memory modeling tool and its application to the design and analysis of future memory hierarchies," in *Proceedings of the 35th Annual International Symposium on Computer Architecture*, ser. ISCA '08. USA: IEEE Computer Society, 2008, p. 51–62. [Online]. Available: https://doi-org.libproxy1.nus.edu.sg/10.1109/ISCA.2008.16

[22] A. Shafaei, Y. Wang, X. Lin, and M. Pedram, "Fincacti: Architectural analysis and modeling of caches with deeply-scaled finfet devices," in *2014 IEEE Computer Society Annual Symposium on VLSI*, 2014, pp. 290–295.

[23] J. Choquette, E. Lee, R. Krashinsky, V. Balan, and B. Khailany, "3.2 the a100 datacenter gpu and ampere architecture," in *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 64, 2021, pp. 48–50.

[24] X. Peng, R. Liu, and S. Yu, "Optimizing weight mapping and data flow for convolutional neural networks on rram based processing-in-memory architecture," in *2019 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2019, pp. 1–5.

[25] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," 2016. [Online]. Available: https://arxiv.org/abs/1609.07843

[26] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "Llama: Open and efficient foundation language models," 2023. [Online]. Available: https://arxiv.org/abs/2302.13971

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 30,2026 at 04:16:59 UTC from IEEE Xplore. Restrictions apply.